
patricia26 Developer Reference Documentation

Release 0.11.0

Gene

Jul 27, 2026

PYTHON API REFERENCE GUIDE:

1	patricia26	1
2	Overview	3
3	Patricia26 Python class	5
4	Performance	7
5	Documentation	9
6	AI Tools	11
7	License	13
8	Python API Reference	15
8.1	Python Patricia26 Class: API Reference	15
9	C API Reference	17
9.1	Patricia26 C-Library: API Reference	17
10	Migrating from PyTricia	21
10.1	Migrating from PyTricia	21
11	Indices and tables	23
	Python Module Index	25
	Index	27

CHAPTER

ONE

PATRICIA26

OVERVIEW

patricia26 has two components: A Patricia Trie library and a Python module using the library. It provides:

- a modern C23 compliant (Linux) version of the the original C-code.
- a Python module generated using Cython that uses the shared c-library.
- an autogenerated API reference guide for both the C and Python code.

The project name made unique to allow the C library to be installed alongside the original and to avoid any conflict with other similar Python modules such as *PyTricia*.

This project was forked from [Dave Plonka's Patricia Library](#) along with some modifications of the core c-code by [Joel Sommers' PyTricia](#). The main change introduced in *PyTricia* changed *node->prefix* from *prefix_t ** to a **prefix_t* thereby reducing the number of *malloc* calls.

The C-code has been tidied some and is now C23 compliant. See the *meson.build* file for more details on compile and link options.

The Python module, *Patricia26*, is uses the shared C library *libpatricia16.so*. The shared library must be installed and available for the Python module to run.

Using Cython to generate the Python module helps keep the module highly performant. Using a shared patricia library allows for:

- C and Python applications to use the same library.
- the core patricia library to be built with stringent compile and link options.

The Cython generated C-code is built using slightly different options than the core c-clibrary since manuallly tidying Cython generated c-code is not terribly practical. As a result, we use somewhat less stringent compile options on that part.

While this is working well for our own use cases, please let us know of any issues should they arise.

The documentation including the C and Python API reference is in *patricia26.pdf* in the *Docs* directory and is also installed in *usr/share/patricia26/*

The Python module, *Patricia26* offers similar functionality to *PyTricia* Both modules are very fast and are built on top of the same original core Patricia C-code. We have used *PyTricia* and found it to be robust and works well.

Patricia26 can be somewhat faster, especially for bulk and cidr lookups.

The API is a little different, but changing from one to the other is quite straightforward.

There is a tentative Python module, *Patricia_int*, that uses integers / *ipaddress* instead of IP and CIDR strings. However, this version is significantly slower, probably due to Python not using simple 32 and 128 bit number representations that can be mapped directly to C, thus necessitating additional conversion overhead that inflicts a pretty big performance penalty. Consequently, we're not using this class and so it has not had the usage and testing that *Patricia26* class has.

PATRICIA26 PYTHON CLASS

The Python module wraps the core patricia C-functions. This is a short highlight on using Pytricia26. Please see the documentation section with the Reference API guide which provides details on the class and its member functions.

Instantiate Class

The class internally keeps both a 32 bit IPv4 tree and a 128 bit IPv6 tree, so instantiating for either is done the same way.

```
from patricia26 import Patricia26
pyt = Patricia26()
```

Saving Values

Values are inserted into the tree either one at a time or from a list of tuples.

For example:

```
pyt[ip_or_cidr] = value
```

or

```
pyt.insert(ip_or_cidr, value)
```

or

```
items = [(ip_or_cidr_1, value_1), (ip_or_cidr_2, value_2), ... ]
pyt.bulk_insert(items)
```

Looking up Values

The best way to lookup the value associated with the longest prefix match (LPM) is to use the optimized lookup() method.

The dictionary subscript syntax (*pyt[ip_or_cidr]*) can also be used for both CIDRs or IP addresses.

When looking up a list of items, *bulk_lookup(list_of_ip_or_cidrs)* is much faster than looping over *pyt.lookup()*. It returns a list of values.

When the value and it's associated LPM prefix are needed, then *lookup_lpm()* returns a tuple of (lpm, value).

Examples:

```
value = pyt.lookup(ip_or_cidr)
value = pyt[ip_or_cidr]
(lpm, value) = pyt.lookup_lpm(ip_or_cidr)
```

with *ip_or_cidr* ~ '10.0.0.2' or '10.1.0.0/22'

```
results = pyt.bulk_lookup(cidrs)
```

is much faster than

```
results = []
for cidr in cidr_list:
    results.append(pyt.lookup(cidr))
```

PERFORMANCE

The benchmark code is available in the *tests* directory. We test lookups on both IP and CIDR inputs using `bulk_lookup()` as well as looping on `lookup()`. Similarly to retrieve both the `lpm` and it's associated value.

Please view these test results as a rough guide only. We recognize there may be ways to improve performance that we may not have taken advantage of.

Doing lookups on plain IP addresses is a bit faster than on CIDR blocks since the latter involves some prefix length calculations. Bulk lookups are substantially faster than using a loop.

Bulk IP lookups can be two to three times as fast as doing the same in a loop: 68 million per second versus 23 million per second. Bulk CIDR lookups at 65 million per second compared to 25 million per sec using a loop.

Bulk lookups should be used with care if the size of the input list is very large due to potential memory pressure. The benchmark tests worked well with 5 million and 50 million IP/CIDRs iterations, with the performance remaining roughly constant. There is variability with different number of iterations but the overall picture remains clear. Tests were run on a laptop with 32 GB of memory.

We should write some additional benchmarks using larger trees, but the high level picture is that these modules are both pretty decent.

Looking up the pair of (LPM prefix, value) using `lookup_lpm()` is a lot slower than retrieving the value alone (8.5 million per second). So, if the `lpm` prefix is not needed, its far better to use `lookup()` or `bulk_lookup()`.

*Pytricia's performance doing value lookups is similar to looping in *Patricia26 and of course slower than bulk_lookup().*

The tests lookup the same IP / CIDR repeatedly which is obviously not terribly realistic. So with that in mind this is sample benchmark output. All benchmarks run with 50 million iterations:

Patricia26 (50,000,000 iterations)				
Method	Input	Lookup	Seconds	Lookups / sec
Bulk	IP	val	0.7300	68,489,951
Bulk	CIDR	val	0.7627	65,552,597
Loop	IP	val	2.1655	23,089,855
Loop	CIDR	val	1.9984	25,019,973
Loop	CIDR	(lpm, val)	5.6163	8,902,680
Loop	IP	(lpm, val)	5.5182	9,060,900
PyTricia (10,000,000 iterations)				
Method	Input	Lookup	Seconds	Lookups / sec
[key]	IP	val	2.0406	24,502,376
get	IP	val	3.2581	15,346,357
get	CIDR	val	3.6105	13,848,587

(continues on next page)

(continued from previous page)

getk	CIDR	(lpm, val)	9.3112	5,369,895
getk	IP	(lpm, val)	8.9636	5,578,110

The Patricia26 tests use `pyt.lookup()`, `pyt.bulk_lookup()` and `pyt.lookup_lpm()`. The lpm lookups carry a burden of additionally providing the prefix string which shows degraded performance compared to simply looking up the value.

The PyTricia tests use `pyt[ip]` and `pyt.get(cidr)` and retrieve the (lpm, value) ~ (lpm = `pyt.get_key()`, `pyt[lpm]`)

When Python uses pickle to save a Patricia26, the class pickles the actual data of (prefix, value) pairs in the tree. The *glue* modes are automatically re-created by the c-clibrary when unpickling and are not saved in the pickled file. This leads to smaller pickled files, and the savings can be very significant. Smaller files naturally means faster IO.

For example, for a tree with 355,951 IPv4 prefixes and their associated values the pickled file size is approximatrly 4MB; if the glue nodes are kept the size would be about 25 MB - so file size is around 16% of what it would be had we retained all the glue nodes.

DOCUMENTATION

The package includes a pre-built PDF guide in *Docs/patricia26.pdf*. The PDF and HTML version can be built by running:

```
make latexpdf ; make latexpdf  
make html
```

Building the docs requires *sphinx* and *hawkmoth*. The documentation includes an auto-generated API Reference Guide that covers both the Patricia C-library and the Patricia26 Python class.

The PDF documentation is installed in *usr/share/patricia26*

AI TOOLS

Anthropic's [Claude](#) played a significant role updating and checking all C-code.

Google's [Gemini](#) was super helpful with building the cython code.

LICENSE

The original patricia code by Dave Plonka is licensed under the University of Michigan and Merit Network. Modifications and additions are licensed by Gene C under the Lesser Gnu Public License version 3 or later.

Modifications made by Joel Sommers <jsommers@colgate.edu> in PyTricia are licensed under the Lesser Gnu Public License version 3 or later.

The license files are available in the *src/share* directory and are installed in the usual place.

PYTHON API REFERENCE

8.1 Python Patricia26 Class: API Reference

This section contains the high-level Python API wrapper for the Patricia Trie. The package includes API stubs file allowing tools such as *mypy* to type check code that uses Patricia26.

Here are some of the important methods:

- `Patricia26()`
Instantiate class
- `Store Value`
`pyt[cidr] = value` `pyt.bulk_insert(cidr_values)`
`cidr_values` is a list of tuples `[cidr, value]`
- `lookup()`
returns the value associated with the input prefix or `None` if no match Can also use dictionary lookup:
`value = pyt.lookup(cidr)` `value = pyt[cidr]`
- `bulk_lookup()`
similar to `lookup()` but takes a list of prefixes and returns a list of values. Faster than looping on `lookup()`
`values = pyt.bulk_lookup(cidrs)`
- `lookup_lpm()`
similar to `lookup()` but returns a tuple of the `(lpm, value)`. Faster than looping on `get_prefix()` plus `lookup()` by maybe 15% - 20%.
- `get_prefix()`
Returns the LPM prefix associated with IP or CIDR string.
- `prefixes()`, `prefixes_v4`, `prefixes_v6`
returns a list of all active prefixes in the tree. The `_v4` / `_v6` variants return list of IPv4 / IPv6 prefixes.
- `freeze()` / `thaw()`
lock/unlock the tree when modifying it.
- `parent()`
Returns the prefix that is the immediate parent of the `lpm`.

- `children()`

returns a list of prefixes that are subnets of the lpm prefix

- `num_prefixes_v4()`, `num_prefixes_v6()`

Returns the number of *active* IPv4 or IPv6 prefixes in the corresponding tree. Note that `len(pyt)` will return the total number of prefixes across both IPv4 and IPv6 trees.

Note that this count includes the active nodes which includes *glue* nodes that patricia automatically inserts when needed, not just those explicitly added.

- `dump_to_file()` / `load_from_file()`

Writes or reads a class instance to or from a file stram object.

Supports the *contains* method.

When checking if prefix is in the tree the *in* operator returns True if the prefix is either an exact match of a tree prefix or is a subnet of one. In contrast, the `has_prefix()` method checks for an exact match to an existing prefix.

There are couple of pytricia compatibility methods that may be useful:

- `keys()` - an alias for `prefixes()`.
- `get_key()` - an alias for `get_prefix()`

For example, Adding and removing key/values to/from the Patricia tree is done using:

```
pyt = Patricia26()
key = '10.0.0.0/24'
val = 'lan-A'
pyt[key] = val
del pyt[key]
```

The next section has the auto-generated API reference.

8.1.1 Python API

C API REFERENCE

9.1 Patricia26 C-Library: API Reference

9.1.1 Data Structures

struct **patricia_tree_tag**

Primary state tracking structure managing a full Patricia Trie instance.

This context block acts as the master anchor for the bitwise radix network, maintaining the top sentinel head node, key length limitations, active telemetry sizing counters, and state lock flags.

Attributes:

head (patricia_node_t*): Pointer to the top-most sentinel entry node of the bitwise matrix. **maxbits (u_int):** The absolute maximum bit depth permissible for key traversal operations (e.g., 32 for IPv4 strings, 128 for IPv6 configurations).

num_active_node (int): Running tracking metric counting the total quantity of populated, valid node elements currently stored inside the tree structure.

frozen (u_short): A configuration bitmask toggle flag (1 or 0) used to freeze the tree layout, preventing subsequent node insertions, deletions, or structural modifications.

struct **prefix_tag**

Unified data model wrapper representing a standardized IP network prefix boundary.

This structure maps structured multi-family properties (IPv4 and IPv6 layouts) into a unified abstract interface. It wraps an anonymous union that can toggle its internal binary byte layout seamlessly depending on the network type.

Attributes:

family (u_short): Address boundary protocol (AF_INET for IPv4 | AF_INET6 for IPv6). **bitlen (u_short):** network bitmask mask length; aka cidr prefix len (e.g., 24 for a /24 block). **ref_count (int):** Ref counter for allocation tracking with memory reclamation. **add (union):** Anonymous data payload container holding the raw binary network address. **add.sin (struct in_addr):** Raw binary structure representing a standard 32-bit IPv4 address. **add.sin6 (struct in6_addr):** Raw binary structure representing a 128-bit IPv6 address.

Support for IPv6 is no longer #if defined away via HAVE_IPV6

struct **patricia_node_tag**

Core node structure within the Patricia Trie array matrix.

Each node represents a bitwise junction point or an actual data payload container inside the routing tree layout.

Attributes:

bit (int): The specific bit index position tested at this tree level. **prefix (prefix_t):** struct with the network IP prefix payload. Can be

NULL for purely intermediate routing nodes.

left (patricia_node_t*): Pointer to the left child node (0-bit branch). right (patricia_node_t*): Pointer to the right child node (1-bit branch). parent (patricia_node_t*): Pointer to the parent node for fast traversal.

Follow pytricia in changing node->prefix to be prefix_t instead of prefix_t * simplifies malloc/free but

9.1.2 Core Functions

patricia_tree_t ***New_Patricia** (int maxbits)

Allocates and initializes a new Patricia Trie structure.

This function builds the base routing tree tracking structure, establishing the maximum depth limit of the radix bit-testing operations.

Args:

maxbits (int): The maximum bitlength depth allowable for keys in this
trie (e.g., 32 for IPv4, 128 for IPv6).

Returns:

patricia_tree_t*: Pointer to the newly allocated and initialized Patricia tree structure, or NULL if memory allocation fails.

prefix_t ***New_Prefix** (int family, void *dest, int bitlen, prefix_t *prefix)

Constructs a structural prefix node container from raw network parameters.

This function translates raw binary representations of network addresses into a standardized prefix block layout compatible with the trie engine.

Args:

family (int): The network address family configuration protocol
(e.g., AF_INET or AF_INET6). **dest (void*):** Pointer to the raw binary network address array. **bitlen (int):** The total routing bitmask bitlength allocation. **prefix (prefix_t*):** Pointer to an allocated destination prefix structure where the result will be populated.

Returns:

int: Returns 1 on successful extraction and construction, 0 if the bitlength exceeds system boundaries, or -1 for unsupported address families.

patricia_node_t ***patricia_lookup** (patricia_tree_t *patricia, prefix_t *prefix)

Searches for a network prefix or inserts it if it is missing.

This is the primary mutating driver of the radix trie. It traverses the bit index paths; if an exact match is missing, a new payload node is created.

Args:

patricia (patricia_tree_t*): Pointer to the base Patricia tree structure. prefix (prefix_t*): Pointer to the target network prefix key layout.

Returns:

patricia_node_t*: Pointer to the newly created or pre-existing matching node structure within the tree matrix, or NULL if allocation fails.

patricia_node_t ***patricia_search_best** (patricia_tree_t *patricia, prefix_t *prefix)

Locates the Longest Prefix Match (LPM) for a given network query.

This function performs an un-mutating longest-match radix search. It maps specific host IP queries back to their broadest covering subnet masks.

Args:

patricia (patricia_tree_t*): Pointer to the base Patricia tree structure. prefix (prefix_t*): Pointer to the target prefix/IP to look up.

Returns:

patricia_node_t*: Pointer to the most specific matching node container covering the target address, or NULL if no covering route exists.

patricia_node_t ***patricia_search_exact** (patricia_tree_t *patricia, prefix_t *prefix)

Searches the Patricia Trie for an exact prefix match.

This function performs a strict bitwise traversal down the radix tree matrix to locate a node that matches the provided network prefix configuration exactly.

Args:

trie (patricia_tree_t*): Pointer to the base initialized Patricia Trie structure. prefix (prefix_t*): The specific IP prefix mask boundary to query.

Returns:

patricia_node_t*: Pointer to the matching node structure if found, or NULL if the target prefix does not exist in the trie matrix.

char ***prefix_toa2x** (prefix_t *prefix, char *buff, int with_len)

Transpiles a binary network prefix layout into a human-readable string.

Converts structural network prefix parameters back into standard ASCII text string notation representations (e.g., "192.168.1.0/24").

Args:

prefix (prefix_t*): Pointer to the source network prefix block structure. buff (char*): Pointer to a destination character array buffer. Must be

adequately sized to handle string overflows.

with_len (int): A boolean flag (1 or 0) toggling whether the subnet mask suffix (like "/24") is appended to the string.

Returns:

char*: Pointer to the destination string buffer (*buff*) containing the formatted ASCII data layout.

TODO:

- should AF_INET case just use inet_ntop()?
- should AF_INET6 handle MAX_THREADS same as AF_INET case?

void **Destroy_Patricia** (patricia_tree_t *patricia, void_fn_t func)

Deallocates and wipes an entire Patricia Trie system memory layout.

This function performs a recursive post-order tree-walk, freeing every node. It permits an optional custom callback to securely purge node data payloads.

Args:

patricia (patricia_tree_t*): Pointer to the Patricia tree system to wipe. func (void_fn_t): An optional custom data-cleanup function pointer to

execute on each node's user data value, or NULL to skip data cleanup.

void **patricia_remove** (patricia_tree_t *patricia, patricia_node_t *node)

Extricates an isolated target node container from the Patricia tree network.

Removes a specific node structural point, rearranging internal branch pointer paths to preserve radix trie sorting and lookup logic.

Args:

patricia (patricia_tree_t*): Pointer to the parent Patricia tree structure. node (patricia_node_t*): Pointer to the specific node container to

remove from the tree array layout.

MIGRATING FROM PYTRICIA

10.1 Migrating from PyTricia

The `Patricia26` API is a little different and this provides a quick overview how to migrate from `PyTricia`.

Instantiate Class

```
from pytricia import Pytricia
pyt_ipv4 = Pytricia(32)
pyt_ipv6 = Pytricia(128)
```

becomes:

```
from patricia26 import Patricia26
pyt_ipv4 = Patricia26()
pyt_ipv6 = Patricia26()
```

Note that there is no bit length argument because `Patricia26` keeps separate IPv4 and IPv6 trees internally.

Store Value

```
pyt[ip_or_cidr] = value
```

becomes:

```
pyt[ip_or_cidr] = value

or

pyt.insert(ip_or_cidr, value)

or

cidr_values = [(ip_or_cidr1, value1), (ip_or_cidr2, value2), ...]
pyt.bulk_insert(cidr_values)
```

Lookup Value

```
val = pyt[ip]
val = pyt.get(cidr)
```

becomes

```
val = pyt.lookup(ip_or_cidr)
```

(continues on next page)

(continued from previous page)

```
or
val = pyt[ip_or_cidr]

or
vals = pyt.bulk_lookup(ip_or_cidr_list)
```

Lookup LPM and Value

```
lpm = pyt.get_key(ip_or_cidr)
val = pyt[lpm]
```

becomes

```
(lpm, val) = pyt.lookup_lpm(ip_or_cidr)

or
lpm = pyt.get_prefix((ip_or_cidr)
val = pyt.lookup(lpm)
```

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

p

patricia26, [16](#)

INDEX

D

`Destroy_Patricia` (*C function*), 19

M

`module`
 `patricia26`, 16

N

`New_Patricia` (*C function*), 18

`New_Prefix` (*C function*), 18

P

`patricia26`
 `module`, 16
`patricia_lookup` (*C function*), 18
`patricia_node_tag` (*C struct*), 17
`patricia_remove` (*C function*), 19
`patricia_search_best` (*C function*), 18
`patricia_search_exact` (*C function*), 19
`patricia_tree_tag` (*C struct*), 17
`prefix_tag` (*C struct*), 17
`prefix_toa2x` (*C function*), 19